Configuration-Driven Integration of Industrial Agents Across Production Scenes

***Abstract*—Industrial production scenes expose different process contexts, signal meanings, and communication protocols, yet task-level agents need a consistent way to read measurements and issue bounded commands. This paper presents a configuration-driven architecture that maps line–product–recipe–run context, acquisition and write-control nodes, protocol drivers, and named operations. Harness adapters connect agent turns to those operations; a shared dispatcher checks authority, while route handlers apply scene-specific limits and approval requirements. Command receipts remain separate from later process observations. We assessed the configured paths in one integrated run using deterministic simulators. Five endpoints in the primary cast-film scene returned 54 samples, and the write-admission checks rejected all 24 negative probes across four writable paths. A second scene returned 51 samples and rejected all nine probes across three writable paths. The 77 workflow checks comprised 76 passes, one warning, and no failures. Three fixed-seed objective cases ended at 96.3%–98.8% of the simulator's offline reference; one case stopped without an update. The run documents the configured mappings and guarded operations in simulation. It used no active language-model loop or physical equipment and did not compare agent harnesses or control baselines.**

***Index Terms*—Industrial agents, industrial communication, industrial informatics, production systems, system integration.**

I. Introduction

INDUSTRIAL production systems expose measurements and control points through devices and protocols configured for each production scene. Before an agent request can reach a controller, the platform must resolve the active line, product, recipe, and run; map the requested variable to the intended control node; and apply the limits for that process. It must also keep an accepted command separate from the later process observation. In the cast-film configuration studied here, an MQTT endpoint carries a line-speed setpoint and film-thickness samples. The shared endpoint does not make the sample a readback of the setpoint or establish that the write caused a process response.

Industrial-agent research addresses process supervision, control generation, service selection, and human-guided automation [1]–[9]. Other work connects agents with digital twins, manufacturing skills, and shop-floor services [10]–[15]. Industrial-informatics architectures have also formalized trust boundaries for digital-twin control, functional blocks for industrial IoT networks, and security services at the fog layer [16]–[18]. Together, these studies address agent reasoning, asset grounding, industrial communication, and guarded execution. The system-integration problem examined here is how to connect agent runtimes to scene-specific signal and protocol maps while applying the same operation checks to task-mediated and direct calls.

We implement a configuration-driven architecture for this integration path. Each scene binds its production context to semantic acquisition and write-control nodes, protocol drivers, objectives, and engineering limits. Harness adapters expose agent turns through a common operation interface. Task-mediated and direct calls enter the same dispatcher; route handlers resolve current context and apply route-specific limits and approval requirements. Service-owned records distinguish command receipts from later process measurements. The work contributes an integration architecture and its implementation, rather than a new control law or agent algorithm.

The paper makes two contributions:

1) A scene-configuration model that binds line–product–recipe–run context to semantic acquisition and write-control nodes, protocol drivers, process objectives, and engineering limits behind a common operation interface.
2) A task-to-device runtime that separates harness execution from scene-specific I/O and routes task and direct calls through shared authorization checks. Route handlers enforce configured limits and approval requirements, while service-owned records preserve the distinction between command receipts and process observations.

Section II reviews related work. Sections III and IV describe the architecture and its execution and governance paths. Section V reports workflow coverage, protocol acquisition and guarded operations, simulator objective cases, and a scripted BOPET task. Sections VI and VII interpret the results and state the scope of the evidence.

Fig. 1 shows how scene context and I/O mappings feed into the shared dispatcher and configured driver.

II. Related Work

The closest work connects agent runtimes to industrial processes, assets, or plant-facing services. These studies differ in the operation boundary they implement and the evidence they report; Table I summarizes representative examples.

A. Industrial Agents and Plant Integration

Industrial-agent studies span production supervision, control generation, PLC programming, human-guided control, and equipment-facing assistance [1], [2], [5]–[9], [19], [20]. Biskupovic et al. focus on service-oriented industrial cyber-physical systems: agents select services using performance and computational-load measures, monitor them online, and reconfigure selection when performance degrades [4]. This line of work studies agent decisions and service adaptation within industrial applications.

Xia et al. connect cognitive, digital-twin, and physical-automation layers and organize twin information as state snapshots, event messages, and plan sequences [10]. Related designs expose factory-module operations through the Model Context Protocol (MCP) with state-based tool filtering, constrain manufacturing plans, or compose semantic recipes from OPC Unified Architecture (OPC UA) skills [3], [11], [12]. Other systems combine agent execution with approval, industrial data, or audit records. Graul et al. ground proposals in the Asset Administration Shell (AAS) and route approved actions through OPC UA Methods, with signed approvals and audit logging [13]. Schall separates advisory language-model agents from process and instrumentation diagram (P&ID)-grounded verification and execution, and evaluates structural checks across five simulated process scenarios [14]; its context includes supervisory control and data acquisition (SCADA) and standard operating procedure (SOP) information. Lin and Chen build a governed digital thread across sensors, the manufacturing execution system (MES), and enterprise resource planning (ERP) for closed-loop monitoring and control on real manufacturing records [15]. These systems address process reasoning, service adaptation, asset grounding, action validation, and audit records. Whereas Biskupovic et al. study service selection and reconfiguration, this work binds replaceable harnesses and task roles to configured production scenes. A shared operation boundary resolves device nodes and keeps command receipts separate from process observations. The table records the mechanisms and evaluation settings reported by each source; the studies were not evaluated under a common benchmark.

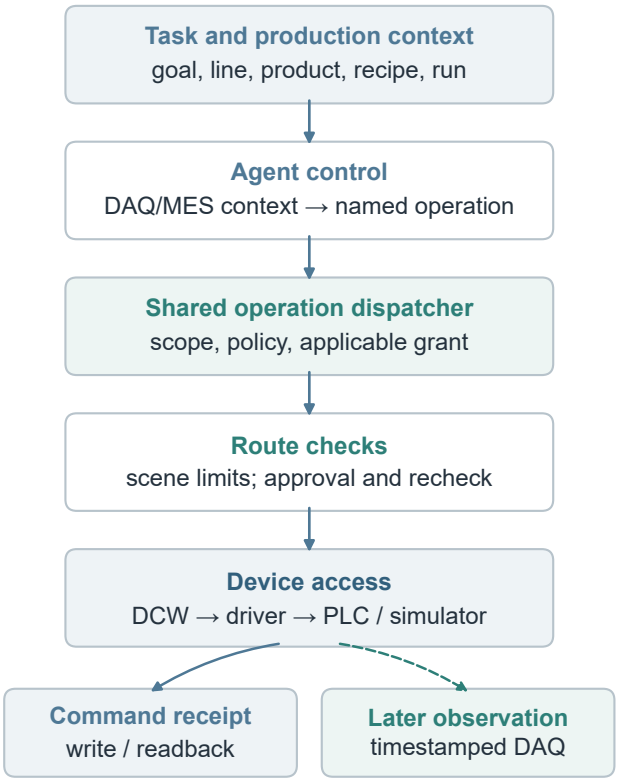


Fig. 1. Task execution from process context to device observation. Data acquisition (DAQ) and manufacturing execution system (MES) services provide context for a named operation. Shared dispatch checks are followed by route-specific limits and, when required, approval with context revalidation. A configured driver accesses the device; the receipt and later process measurement remain separate records.

Related industrial-informatics work also informs this design. Gehrmann and Gunnarsson describe trust boundaries and synchronization requirements for digital-twin-based industrial control and evaluate a protected-update proof of concept [16]. Mahmood et al. organize special-purpose industrial IoT functions around network requirements, while Sengupta et al. distribute security services across fog and device layers and report simulation and testbed evidence [17], [18]. These works address communication, security, and functional decomposition. The operation checks described here complement safety-aware decision support [21] and do not replace configured plant protection.

B. Production Context, Agent Interfaces, and Automation

Industrial integration also relies on abstractions that are not specific to agent architectures. The Model Context Protocol (MCP) specifies interoperable tool discovery and call mechanisms [22]. The platform dispatcher's operation-level scope, policy, grant, and binding checks are distinct from MCP transport authorization. ISA-88 and ISA-95 describe recipes and production operations [23], [24]. Run-to-run methods connect process observations to successive production runs [25]. Human–automation research also formalizes the allocation of information acquisition, analysis, decisions, and actions between people and automation [26]. At the agent level, AgentSpec and CaMeL address tool behavior and capability-based information flow [27], [28]. These standards and concepts inform how the design handles production context, tool interaction, human allocation, and information-flow boundaries; the system does not implement all of them. The selected studies address these concerns through separate interfaces and validation layers.

III. System Design

The architecture links an agent collaboration runtime to a production-scene configuration. The runtime supplies task context and scoped operations. The scene configuration maps process semantics to acquisition and control nodes, drivers, targets, and guards. Named operations carry requests from the runtime to shared dispatch and governed device access. Task execution remains separate from operation dispatch, and command receipts remain distinct from measured process responses.

A. Design Rationale and Requirements

The design must preserve process meaning across heterogeneous drivers, allow harness changes without altering the operation contract, and check current authority and process limits before a write. It must also keep command acceptance distinct from subsequent process observations. Scene configuration, registered harness adapters, shared dispatch, route handlers, and service-owned records provide these boundaries.

TABLE I
REPORTED OPERATION MECHANISMS AND EVALUATIONS IN SELECTED PLANT-FACING STUDIES. EVALUATION SETTINGS AND OUTCOMES ARE SOURCE-SPECIFIC; THIS TABLE DOES NOT COMPARE PERFORMANCE ACROSS STUDIES.

System	Reported operation or integration mechanism	Reported evaluation
Xia et al. [10]	Cognitive, twin, and physical layers; twin snapshots, event messages, and plan sequences	Robotic automation and process-simulation cases
Biskupovic et al. [4]	Agent-based selection and online reselection of services using performance and computational-load metrics	Reported adaptability and performance improvement in service-oriented industrial cyber-physical systems
Köhle et al. [3]	Injected live state and state-based filtering; MCP-to-OPC UA operations and Message Queuing Telemetry Transport (MQTT) coordination	Nine simulated production challenges
Vieira da Silva et al. [11]	Constrained multi-step plans over MCP manufacturing functions	Laboratory multi-step process
Köhle et al. [12]	Semantic recipes composed from OPC UA skills; virtual commissioning	Unity assembly-workstation proof of concept
Graul et al. [13]	AAS-grounded proposals, signed human approval, PLC-validated OPC UA Methods, and immutable audit log	Two laboratory PoCs; single scripted runs
Lin and Chen [15]	Governed sensor–MES–ERP digital thread with SOP-, risk-, and capability-aware action gating	25,275 real records; operational metrics
Schall [14]	P&ID-, SCADA-, and SOP-grounded proposal verification with gated execution	Five simulated scenarios; 43 proposals and 50 LLM runs
This work	Common task-operation interface with registered harness adapters; scene-bound context and acquisition/write-node mappings; bounded named operations; separate command and observation records	One deterministic integrated simulator run; distinct protocol and guard checks, fixed-seed objective cases, and a scripted BOPET integration trace

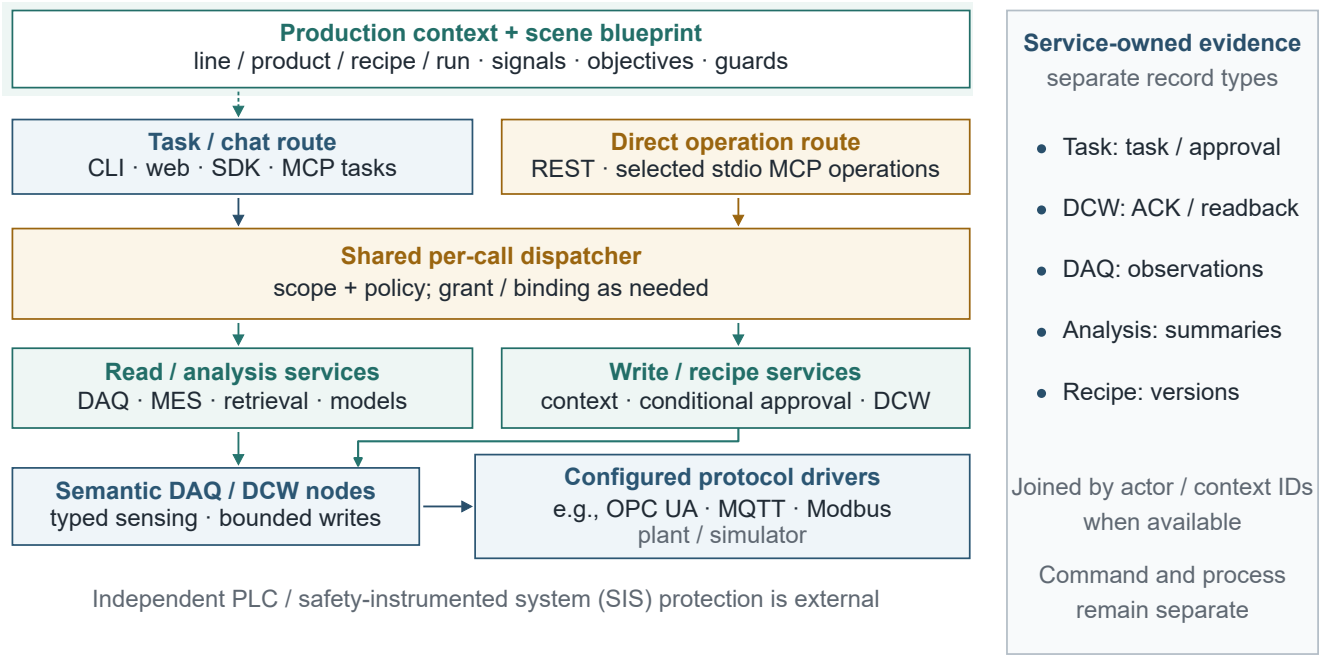


Fig. 2. Architecture and record ownership. Task/chat routes enter the collaboration runtime; direct REST and stdio-MCP calls enter shared dispatch checks without creating a task. The dashed line marks the configuration link. Write routes apply interval admission and request approval when configured. Recipe dispatch omits the previous-recipe window. The shared dispatcher always checks scope and channel policy; it also checks grants and resource bindings when the operation requires them. Owning services retain distinct records; plant controllers and safety systems retain equipment interlocks.

The registry allows agent engines and protocol drivers to be added without changing named operations. Section V evaluates selected configured simulator paths.

Commissioning links each line to its products, recipes, active-run rules, signal meanings and units, driver mappings, and admissible limits. For a task turn, the context builder assembles the current line–product–recipe–run state and relevant process summaries. Before a write, the handler resolves live

state again; prompt context informs a proposal but does not authorize it. Direct operation calls resolve the current context at invocation.

B. Layered Architecture

Fig. 2 shows the access routes, shared dispatch path, production services, semantic I/O, configured drivers, and ownership of records. Task and chat routes enter the collaboration runtime. It contains workspaces, channels, task queues, shared memory, mailboxes, task templates, and harness adapters. A *task route* is an execution path through this runtime; a *harness adapter* connects an agent engine to its task interface. Multiple registered agents can work through shared tasks and channel context, but each agent's available tools are constrained by its binding and channel policy.

Direct REST calls and the stdio-MCP operation bridge bypass task execution and enter the shared dispatch block (upper-right route in Fig. 2). They create neither a task nor a harness turn. MCP management and task methods remain on their service routes.

The scene configuration supplies the active production context, semantic node mappings, objectives, and guards to the operation path; the dashed line in Fig. 2 marks this link. The service band provides data acquisition, write control, analysis, retrieval, and optional model services. Field I/O maps semantic nodes to built-in or plugin drivers for a plant or simulator. Each owning service retains its task, command, observation, or configuration records and associates them with the actor and production context where supported.

C. Cross-Module Execution Flow

Fig. 3 shows the task-to-device sequence. The runtime gives the registered adapter a scoped tool catalog and assembles the current task context. For each returned call, the shared dispatcher evaluates its declared scope, channel-policy, grant, and binding checks before selecting a handler. That handler resolves live production state and applies the route's guards, active-run requirements, limits, and approval rules before device I/O. The result returns to the caller; the owning services retain the operation receipt and any later process observations separately.

For an industrial operation u by agent a in channel c, define the shared dispatch condition as

$$\mathcal{A}(a, c, u) = \bigwedge_{X \in \mathcal{C}(u)} X(a, c, u), \tag{1}$$

where S, P, G, and B denote invocation scope, current channel policy, applicable industrial grant, and the agent's binding to referenced resources, respectively. $\mathcal{C}(u) \subseteq \{S, P, G, B\}$ is the set of predicates declared for operation u; it always contains S and P. The grant predicate G is included when the resolved channel, target line, and owner/grant policy require it; explicit exemptions omit it. The binding predicate B is included when the operation declares resource references. A declared no-reference operation omits B. Each included check must pass before handler selection. The policy predicate includes channel tool selection, profile, and duty-separation checks where configured. After these predicates pass, the selected route applies its guards, active-run checks, admission limits, and any required approval. If approval is required, the handler revalidates its context before device dispatch.

For a direct REST call, the service establishes a and c before applying these checks. An agent token must match the requested agent identifier; an authenticated user must be the channel owner or an administrator. The manager loads the channel-agent record and rechecks scope and policy, followed by any applicable grant and binding checks. Handlers resolve live production state from request identifiers and owning services.

Direct REST and stdio-MCP operation calls bypass the task runtime but enter the same shared dispatcher. Section IV specifies route-level admission, timing, approval, and evidence rules.

IV. Module Functions and Integration

Section III defines the shared dispatch contract. This section focuses on the checks that follow it: scene-specific write admission, conditional approval, and the records returned by each route.

A. Scene-Specific Write Admission

After the shared dispatch checks pass, the selected handler applies the limits configured for the target scene and control. Authorization and value admission are separate checks. The target band describes a desired process outcome; it does not define the admissible write interval. A candidate may satisfy all write constraints while the measured process value remains outside the target band.

For an online node write, the admissible value interval is

$$\mathcal{I}_n^{\text{online}}(t) = \mathcal{H}_n \cap \mathcal{L}_n^{\text{base}} \cap \mathcal{L}_n^{\text{prod}}(t) \cap \mathcal{L}_n^{\text{rec}}(t), \tag{2}$$

where $\mathcal{H}_n$ is the hard engineering range, and $\mathcal{L}_n^{\text{base}}$, $\mathcal{L}_n^{\text{prod}}$, and $\mathcal{L}_n^{\text{rec}}$ are the baseline, active-product, and active-recipe limits. These numerical bounds are distinct from the scope, policy, grant, and binding predicates in Eq. (1). Missing optional limits impose no restriction. An empty intersection rejects the request and identifies the excluding layer. Recipe dispatch checks a replacement recipe against the hard, baseline, and product limits rather than the previous recipe's window. Step, interval, and hold-window limits apply when configured. These application-level checks complement, but do not replace, equipment interlocks.

B. Approval, Execution, and Evidence

Routes add timing checks and request approval when the operation and its configuration require it. Outside benchmark mode, online agent and manual node writes must be at least 60 s apart. Recipe trials follow a per-line cadence of 300 s by default; their intervals start at approval. Manual approval binds an actor and proposed operation to candidate values, applicable limits, expiry, and the active line–product–recipe–run tuple. After approval, the handler rechecks authority and compares the current tuple with the approved tuple before device-write dispatch. Single-parameter waits expire after 180 s by

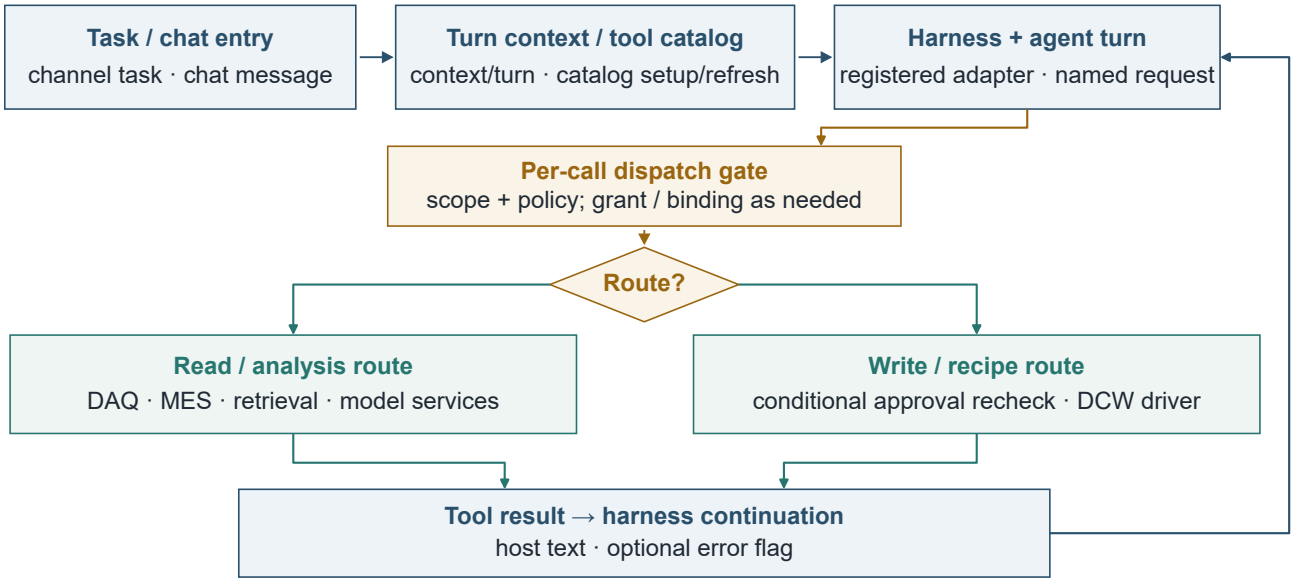


Fig. 3. Registered-harness task flow. The runtime invokes the adapter with a scoped catalog. Scope and channel policy are checked for every call; grant applies to non-exempt industrial operations, and binding applies when resource references are declared. Failed checks return tool errors. The read path retrieves data. The write path applies route guards, conditional approval and its context revalidation, write admission, and receipt grading; a later process observation is separate. Results return to the harness. Device-level interlocks remain with the configured plant-control layer.

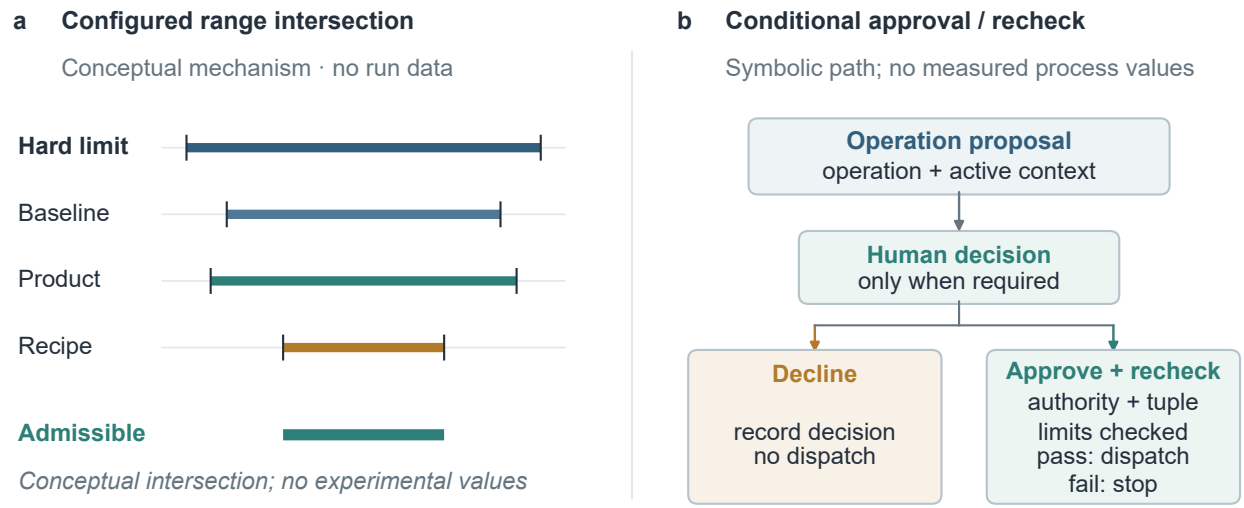


Fig. 4. Admission and approval boundaries. (a) Conceptual intersection of the hard, baseline, product, and recipe bounds in Eq. (2); horizontal positions are schematic and encode no experimental values. (b) When approval is required, a decline prevents dispatch; approval is followed by authority, context, and limit rechecks.

default. Recipe-dispatch approvals have a 30-minute window. A changed recipe definition requires a new `recipe_apply` proposal.

Fig. 4(a) illustrates the four-layer interval in Eq. (2) as a conceptual intersection of configured bounds; its horizontal positions do not represent measured or run-derived values. Panel (b) shows the decision and rechecks when approval is required.

A `recipe_trial` illustrates this sequence. The handler checks the bound recipe, active run, candidate parameters, and route timing. If approval is required, it requests approval and revalidates the request before submitting a bounded batch through the configured driver. It returns a receipt for each parameter. A trial does not save or apply a recipe version; the caller can retrieve later DAQ observations separately before updating, applying, or rolling back the recipe. The driver registry maps semantic controls to configured protocol drivers, so operation arguments do not contain protocol-specific details.

Receipts record what happened to a command; observations record what followed in the process. Command results are

TABLE II
WORKFLOW CHECKS FROM THE INTEGRATED PIPELINE RUN. COUNTS ARE PASS/WARN/FAIL; ALL 17 PHASE GATES PASSED.

Workflow block	Gates	Checks (P/W/F)	Evidence exercised
Bootstrap, model setup, and scene provisioning	3	11/0/0	Runtime checks and five-protocol node configuration
Acquisition, governed operations, and service records	8	30/0/0	Sampling, negative probes, approval, recipe, and task routes
Seeded objective and multimodal acquisition checks	2	7/0/0	Three fixed seeds and vector/image acquisition
Second-scene configuration and backstop checks	2	16/0/0	Configuration-only remapping and bounded recovery
Platform subsystem checks	1	3/0/0	Service and registry checks
BOPET scene and scripted task	1	9/1/0	Scene provisioning and three setpoint writes; one semantic-card warning
Total	17	76/1/0	No failed checks

graded as readback-verified, transport-acknowledged, or unverified. A readback mismatch can trigger up to three reads at 2 s intervals, and recipe batches retain verification outcomes for each parameter. A readback-verified result means the returned value matched the request. A transport acknowledgement alone does not establish that the device applied the value, and an unverified result does not establish a matching readback. DAQ stores later timestamped process observations, while recipe history, approvals, and task records remain with their owning services. Identifiers and timestamps can link these records when available, but a later observation alone does not prove that the command caused the measured change. Section V evaluates selected parts of this evidence chain under the stated test conditions.

V. EXPERIMENTAL EVALUATION

We evaluated the configured integration path in one integrated pipeline run. The run comprised workflow checks, protocol acquisition and guarded writes, an approval-enabled operation, fixed-seed simulator cases, and a scripted BOPET task. The results below are grouped by the capability exercised; each group is a distinct check from the same run.

A. Run Design and Conditions

The run exercised configured task-to-device paths in local deterministic simulators. A deterministic OpenCode tool adapter invoked platform operations; no language-model loop or physical equipment was used. The platform and simulator were already running, and the run reused shared platform data. The run identifier was `20261031034253-1dbo`; Supplementary File S6 records its reproduction command, source commit, and selected evidence.

All 17 phase gates passed. Of the 77 checks, 76 passed and one raised a warning; none failed. The warning came from the BOPET semantic-card check in P10, while its phase gate passed. Table II summarizes the checks by workflow block.

B. Protocol Acquisition and Guarded Operations

The primary cast-film scene used five simulated endpoints: Modbus TCP, Modbus RTU, OPC Unified Architecture (OPC UA), Message Queuing Telemetry Transport (MQTT), and HTTP. The five acquisition nodes returned 54 samples. Four endpoints also had write-control paths; each received six negative probes, and the configured write-admission checks rejected all 24. Each writable path also recorded an accepted legal write. Modbus RTU provided acquisition only and is excluded from the write-probe count. On the MQTT endpoint, `linespeed-sp` maps to the write node and `thick` to acquisition. That endpoint returned six thickness samples and recorded one accepted write. This acquisition/write mapping exercises distinct signals on the same protocol endpoint.

The run also exercised the human-in-the-loop approval path (P4c). A Line 2 request for 145.5 remained pending until approval returned status 200; the pending write was then released. The simulated PLC readback was 146, and the recorded pending-to-approval interval was 452 ms, including polling.

In the same run, the second-scene (`film-line`) configuration used the same five protocols and returned 51 acquisition samples. Its three writable routes each recorded one accepted legal write and rejected three negative probes. The run report recorded no false block or code change. Table III compares acquisition and negative-probe counts across the two scene configurations.

C. Deterministic Tool-Route Execution

The deterministic tool adapter executed the task route on four writable cast-film paths. Each path completed three setpoint steps toward the midpoint of its configured window. Modbus TCP and OPC UA returned actuator readbacks after each step; MQTT and HTTP recorded accepted writes and DAQ checks but did not expose actuator readback. Table IV lists the commanded values and checks. Together, these traces document task-to-driver execution across the four configured write protocols.

D. Fixed-Seed Simulator Objective Cases

The run included three fixed-seed objective cases evaluated against the simulator's offline reference, $J^* = 89.894$. The runner defines J_{end} as the mean of the last two valid trajectory scores. If a case stops before a second score is available, J_{end} is the sole valid score. Table V also lists J_{last}, the final individual trajectory score. The J_{end}/J^* ratios ranged from 96.3% to 98.8%, with a mean of 97.8%.

The simulator score is $J = 55J_{\text{thick}} + 25J_{\text{quality}} + 8J_{\text{energy}} + 7J_{\text{throughput}}$. The thickness term scores deviation from $50 \pm 2\,\mu\text{m}$ with soft degradation outside that band; the quality term reflects defect rate, the energy term screw speed, and the throughput term line speed. The score is defined only for melt temperature in 195–225 °C and pressure no greater than 22 MPa.

TABLE III
ACQUISITION AND REJECTED NEGATIVE PROBES IN THE CAST-FILM AND `film-line` CONFIGURATIONS FROM ONE RUN. "–" MARKS AN ACQUISITION-ONLY ENDPOINT.

Protocol	Cast-film samples	Rejected probes	`film-line` samples	Rejected probes
Modbus TCP	9	6/6	12	3/3
OPC UA	11	6/6	12	3/3
MQTT	6	6/6	5	3/3
HTTP	13	6/6	11	–
Modbus RTU	15	–	11	–
Total	54	24/24	51	9/9

TABLE IV
DETERMINISTIC OPENCODE TASK-ROUTE TRACE FROM THE INTEGRATED RUN. EACH SEQUENCE APPROACHES THE CENTER OF ITS ROUTE WINDOW; VALUES USE THE ROUTE'S CONFIGURED UNITS. READBACK AND DAQ COLUMNS COUNT SUCCESSFUL CHECKS AMONG THREE STEPS; "NOT EXPOSED" MEANS THE DRIVER DID NOT RETURN AN ACTUATOR READBACK.

Protocol	Setpoint sequence	Readback	DAQ
Modbus TCP	194.96 → 197.48 → 200.00	3/3	3/3
OPC UA	144.6 → 147.3 → 150.0	3/3	3/3
MQTT	91.4 → 93.2 → 95.0	Not exposed	3/3
HTTP	0.946 → 0.973 → 1.000	Not exposed	3/3

TABLE V
FIXED-SEED OBJECTIVE OBSERVATIONS FROM THE INTEGRATED SIMULATOR RUN. J_0 IS THE INITIAL SCORE; J_{end} IS THE MEAN OF THE LAST TWO VALID OBSERVATIONS, OR THE SOLE VALID OBSERVATION FOR AN EARLY STOP; J_{last} IS THE FINAL INDIVIDUAL TRAJECTORY SCORE.

Seed	J_0	J_{end}	J_{last}	J^*	Ratio	Iter.	Writes
42	66.271	88.775	88.853	89.894	98.8%	2	2
43	86.542	86.542	86.542	89.894	96.3%	0	0
44	63.981	88.384	88.540	89.894	98.3%	2	2

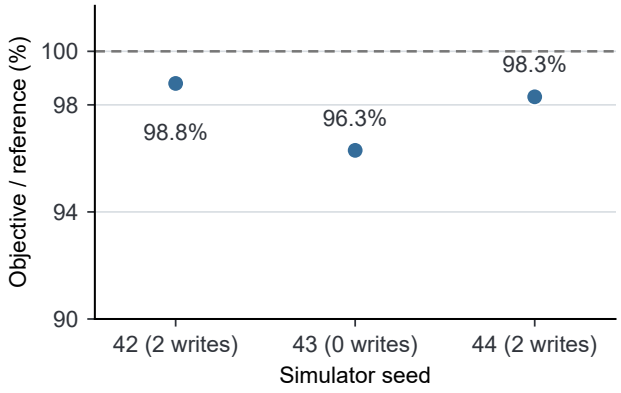


Fig. 5. The three fixed-seed runner-defined end-score ratios (J_{end}/J^*) were 96.3%–98.8% of the simulator's offline objective reference. Here J_{end} is the mean of the last two valid scores, or the sole valid score when a case stops earlier; J_{last} is the final individual trajectory score. The dashed line marks 100%; the vertical axis begins at 90% to show the case-level spread. Seed 43 stopped before any update.

Seed 43 began inside the configured stopping region and therefore required no iterations or writes; seeds 42 and 44 each used two writes. The offline reference is a simulator-defined target. Fig. 5 shows the three case-level ratios; it does not imply a distribution estimate.

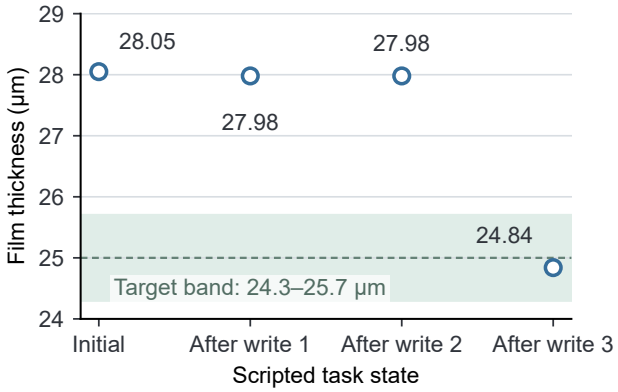


Fig. 6. One scripted BOPET task recorded an in-band terminal thickness of $24.84\,\mu$m after three writes. The shaded area marks the configured 24.3–$25.7\,\mu$m target band. The task-board task completed; its associated parameter-change/optimization record remained open and unjudged.

E. Scripted BOPET Integration Trace

One scripted task in the run issued three setpoint writes to three controls on a biaxially oriented polyethylene terephthalate (BOPET) simulator scene. The thickness readings were 28.05, 27.98, 27.98, and $24.84\,\mu$m; the final reading was inside the configured $25.0 \pm 0.7\,\mu$m band. The task board marked the task COMPLETED; its associated parameter-change record remained open and unjudged. The BOPET phase also raised the single semantic-card warning. Fig. 6 shows this task separately from the fixed-seed objective cases.

VI. DISCUSSION

A. Interpretation

In the integrated run, both scene configurations used the shared operation boundary while retaining their own signal and route mappings. Five protocol endpoints returned acquisition samples, and the tested negative probes were rejected in both scenes. The result supports integration for these configurations; it does not show that an arbitrary plant can be connected without commissioning or route-specific setup.

The deterministic OpenCode task route completed three-step sequences on four writable cast-film routes with the model loop disabled. Modbus TCP and OPC UA returned actuator readbacks; MQTT and HTTP recorded accepted operations and DAQ checks without readback. The second scene was exercised for acquisition and guard checks only, so this run does not establish harness portability or a cross-scene write-response chain.

The objective cases and BOPET task are separate simulator activities from the same run. The former were compared with an offline simulator reference; the latter ended inside its configured thickness band, while its associated parameter-change record remained open and unjudged. Neither activity provides a matched controller comparison or shows that a write caused a process change.

Prior work grounds industrial actions in digital-twin state, asset models, process diagrams, or supervisory checks [13], [14], [16]. This paper focuses on the interface between task or harness execution, scene-specific signal and driver mappings, and the shared device-operation path. The run tested selected links across that interface. It did not measure whether configuration reduces integration time, maintenance effort, or operational risk; those questions require matched deployments and workload measurements.

B. Scope and Limitations

The evidence comes from one deterministic simulator run, with the platform and simulators already running. There was no active language-model loop, cold-start commissioning, matched comparison of task and direct calls, multi-harness test, approval ablation, controller baseline, or physical equipment. The three seeds are cases within that run, not independent system repetitions; seed 43 stopped before any update, so they do not establish convergence across initial conditions.

The negative probes exercise recorded software guards only. They do not measure approval effectiveness or establish plant safety, and the missing denominator of legal-write attempts prevents an estimate of false blocks. Supplementary File S6 retains latency summaries as diagnostics because the run did not preserve event-level timings. Its delivery manifest was created after execution and does not establish that the runtime configuration was frozen.

To test effectiveness, a follow-up should repeat the run with a frozen configuration, record complete request-to-readback traces, and compare matched controller and approval-policy conditions. Hardware-in-the-loop evaluation can then test whether the software results transfer to equipment.

VII. Conclusion

We presented a configuration-driven architecture that connects task-level industrial agents to scene context, signal mappings, and device routes through a shared governed interface. In one deterministic simulator run, configured acquisition and write-guard paths operated in two scenes; 76 of 77 workflow checks passed, with one warning. These results document the integration in simulation; portability across harnesses and control effectiveness remain open questions.

Acknowledgment

The authors used AI-assisted tools for language editing and take responsibility for the manuscript. Funding and author identities are withheld in this review copy.

Author Contributions

Individual CRediT contributions are declared in the submission system and withheld in this review copy.

Declaration of Competing Interest

The authors declare no known competing financial interests or personal relationships that could have appeared to influence this work.

Data Availability

Supplementary File S6 contains selected run records, source data, and figure inputs. Its checksums are delivery-time hashes, not runtime signatures; it contains no physical-plant data.

References

[1] Y. Xia, N. Jazdi, J. Zhang, C. Shah, and M. Weyrich, "Control industrial automation system with large language model agents," in *Proc. IEEE 30th Int. Conf. on Emerging Technologies and Factory Automation (ETFA)*, 2025, pp. 1–8, doi: 10.1109/ETFA65518.2025.11205539.

[2] K.-C. Huang, H.-T. Lin, C.-H. Lin, and M.-T. Huang, "Large language model integration in industrial cyber-physical systems: A multi-modal architecture for automated control generation," in *Proc. Int. Conf. on Control, Automation and Diagnosis (ICCAD)*, 2026, pp. 1–6, doi: 10.1109/ICCAD69956.2026.11643124.

[3] K. Köhle, D. Anicic, T. A. Runkler, and R. Graf, "LLM-driven multi-agent control for skill-based smart manufacturing," in *Proc. IEEE 31st Int. Conf. on Emerging Technologies and Factory Automation (ETFA)*, 2026, pp. 1–8, doi: 10.1109/ETFA68377.2026.11725257.

[4] A. Biskupovic, A. Villalonga, F. Castaño, R. E. Haber, and F. Núñez, "Agent-based supervision for service-oriented industrial cyber–physical systems," *IEEE Trans. Ind. Inform.*, vol. 21, no. 3, pp. 2719–2728, 2025, doi: 10.1109/TII.2024.3514178.

[5] Z. Liu *et al.*, "Agents4PLC: Automating closed-loop PLC code generation and verification in industrial control systems using LLM-based agents," *IEEE Trans. Softw. Eng.*, vol. 52, no. 5, pp. 1672–1687, 2026, doi: 10.1109/TSE.2026.3667895.

[6] R. Wu, J. Ai, and T. Li, "InstructMPC: A human-LLM-in-the-loop framework for context-aware control," in *Proc. IEEE 64th Conf. on Decision and Control (CDC)*, 2025, pp. 172–179, doi: 10.1109/CDC57313.2025.11312156.

[7] J. Lim and I. Kovalenko, "A large language model-enabled control architecture for dynamic resource capability exploration in multi-agent manufacturing systems," in *Proc. IEEE 21st Int. Conf. on Automation Science and Engineering (CASE)*, 2025, pp. 2088–2095, doi: 10.1109/CASE58245.2025.11163802.

[8] Z. Zhao *et al.*, "A large language model-based multi-agent manufacturing system for intelligent shopfloors," *Adv. Eng. Inform.*, vol. 69, 2026, art. no. 103888; doi: 10.1016/j.aei.2025.103888.

[9] B. Hofmann, N. Piechulek, S. Kreitlein, J. Franke, and P. Bruendl, "Beyond touch-based human-machine interface: Control your machines in natural language by utilizing large language models and OPC UA," *arXiv preprint arXiv:2510.11300*, 2025. [Online]. Available: https://arxiv.org/abs/2510.11300v2

[10] Y. Xia, N. Jazdi, and M. Weyrich, "An architecture for integrating large language models with digital twins and automation systems," in *Proc. IEEE 30th Int. Conf. on Emerging Technologies and Factory Automation (ETFA)*, 2025, pp. 1–8, doi: 10.1109/ETFA65518.2025.11205636.

[11] L. M. Vieira da Silva, A. Köcher, and F. Gehlhoff, "Beyond formal semantics for capabilities and skills: Model context protocol in manufacturing," in *Proc. IEEE 30th Int. Conf. on Emerging Technologies and Factory Automation (ETFA)*, 2025, pp. 1–4, doi: 10.1109/ETFA65518.2025.11205601.

[12] K. Köhle, D. Anicic, T. A. Runkler, K. Dorofeev, A. S. Thuluva, and R. Graf, "Virtually commissioned semantic recipes based on OPC UA skills," in *Proc. IEEE 8th Int. Conf. on Industrial Cyber-Physical Systems (ICPS)*, 2025, pp. 1–6, doi: 10.1109/ICPS65515.2025.11087832.

[13] C. Graul, W. Żarski, D. Mikołajewski, and I. Rojek, "Ai agents in industry 4.0: AAS–OPC UA–LLM architecture as the foundation of intelligent manufacturing systems in the context of industrial enterprise implementation," *Applied Sciences*, vol. 16, no. 11, p. 5428, 2026, doi: 10.3390/app16115428.

[14] D. Schall, "Safe integration of large language models into industrial process control: A multi-agent architecture with P&ID-grounded validation," *Autonomous Intelligent Systems*, vol. 6, 2026, article no. 14; doi: 10.1007/s43684-026-00136-1.

[15] S.-C. Lin and C.-Y. Chen, "An integrated framework featuring policy-governed agentic AI for closed-loop manufacturing control with multi-source sensor–MES–ERP," *The International Journal of Advanced Manufacturing Technology*, vol. 143, pp. 6405–6428, 2026, doi: 10.1007/s00170-026-17806-2.

[16] C. Gehrmann and M. Gunnarsson, "A digital twin based industrial automation and control system security architecture," *IEEE Trans. Ind. Informatics*, vol. 16, no. 1, pp. 669–680, 2020.

[17] N. H. Mahmood, G. Berardinelli, E. J. Khatib, R. Hashemi, C. de Lima, and M. Latva-aho, "A functional architecture for 6g special-purpose industrial iot networks," *IEEE Trans. Ind. Informatics*, vol. 19, no. 3, pp. 2530–2540, 2023.

[18] J. Sengupta, S. Ruj, and S. Das Bit, "A secure fog-based architecture for industrial internet of things and industry 4.0," *IEEE Trans. Ind. Informatics*, vol. 17, no. 4, pp. 2316–2324, 2021.

[19] M. S. Gill, J. Vyas, A. Markaj, F. Gehlhoff, and M. Mercangöz, "Leveraging LLM agents and digital twins for fault handling in process plants," in *Proc. IEEE ETFA*, 2025, pp. 1–8, doi: 10.1109/ETFA65518.2025.11205597.

[20] M. Waseem, K. Bhatta, C. Li, and Q. Chang, "Pretrained LLMs as real-time controllers for robot operated serial production line," *Mach. Learn.: Eng.*, vol. 2, no. 1, 2026, art. no. 015006; doi: 10.1088/3049-4761/ae4ee8.

[21] X. Jiang *et al.*, "Agentic AI for safety-aware process monitoring and fault diagnosis: A review," *Processes*, vol. 14, no. 13, 2026, art. no. 2112; doi: 10.3390/pr14132112.

[22] Model Context Protocol contributors, "Model Context Protocol: Tools," Specification, 2026, accessed: Oct. 20, 2026. [Online]. Available: https://modelcontextprotocol.io/specification/2026-07-28/server/tools

[23] ISA, "ISA-88.00.01-2010: Batch control—part 1: Models and terminology," International Standard, 2010.

[24] ——, "ANSI/ISA-95.00.01-2025 (IEC 62264-1 mod): Enterprise-control system integration—part 1: Models and terminology," International Standard, 2025.

[25] J. Moyne, E. del Castillo, and A. M. Hurwitz, Eds., *Run-to-Run Control in Semiconductor Manufacturing*, 1st ed. Boca Raton, FL: CRC Press, 2000.

[26] R. Parasuraman, T. B. Sheridan, and C. D. Wickens, "A model for types and levels of human interaction with automation," *IEEE Trans. Syst., Man, Cybern., Part A*, vol. 30, no. 3, pp. 286–297, 2000, doi: 10.1109/3468.844354.

[27] H. Wang, C. M. Poskitt, and J. Sun, "AgentSpec: Customizable runtime enforcement for safe and reliable LLM agents," in *Proc. IEEE/ACM 48th Int. Conf. on Software Engineering (ICSE)*, 2026, pp. 2938–2950, doi: 10.1145/3744916.3764546.

[28] E. Debenedetti *et al.*, "Defeating prompt injections by design," 2025, arXiv preprint arXiv:2503.18813. [Online]. Available: https://arxiv.org/abs/2503.18813